National University of Singapore

CS2030S: Programming Methodology II
AY 2023-2024 – Semester 1

Final Assessment

INSTRUCTIONS TO CANDIDATES

1. This assessment paper contains **SEVENTEEN (17)** questions and comprises **SIXTEEN (16)** printed pages.
2. Write all your answers in the space provided in the answer sheet.
   - Any answer not in space provided will not be graded.
3. The total mark for this assessment is 100.
4. This is an **OPEN BOOK** assessment. You are only allowed to refer to hardcopies materials.
5. All questions in this assessment paper uses Java 17.
6. All codes in this assessment paper are compiled with `-Xlint:unchecked` and `-Xlint:rawtypes`.
7. Answer **ALL** questions.
8. The relevant APIs (`Stream`, `CompletableFuture`) are given on page 13 to 16.

Marks

Questions	Points
1 - 6	18
7 - 12	24
13	15
14	8
15	8
16	12
17	15
Total	**100**

Good Luck!

Section I: MCQ and MRQ

The next **SIX (6)** questions are MCQ. There is only one correct answer. All questions are **independent** of each other. Each question is worth 3 marks.

Shades the bubble in the answer sheet accordingly. Please use pencil. If you need to change your answer, **completely erase** your previous answer before shading another answer.

1. (3 points) Which one of the following concepts is **NOT** needed to harness the power of polymorphism?

   (A) Subclass
   (B) Method overloading
   (C) Method overriding
   (D) Superclass
   (E) *None of the above*

2. (3 points) Which one of the elements below is **NOT** part of the stack/heap diagram used in this course?

   (A) Call frame
   (B) Arrows
   (C) Metaspace
   (D) Stack and Heap
   (E) *None of the above*

3. (3 points) Which one of the following statements below is **true** with respect to Java abstract class.

   (A) An abstract class can be instantiated.
   (B) We can instantiate an anonymous class extending an abstract class.
   (C) An abstract class must have abstract method.
   (D) All abstract methods in an abstract class is automatically declared `public`.
   (E) *None of the above*

4. (3 points) Which one of the following methods on stream is expensive to invoke on *ordered* stream when run in parallel?

   (A) `map`
   (B) `filter`
   (C) `limit`
   (D) `flatMap`
   (E) *None of the above*

Continue on the next page.

For **Question 5 and 6** below, consider the following property of a **pure** function F and an element e.

1. e is the identity of F (*i.e.,* $F(e, x) = x$).
2. F is associative (*i.e.,* $F(x, F(y, z)) = F(F(x, y), z)$).
3. F is commutative (*i.e.,* $F(x, y) = F(y, x)$).

Also consider the following `reduce` method introduced for stream (*see API for more information*).

```
T reduce(T identity, BinaryOperator<T> accumulator)
```

Lastly, if there are multiple answers, choose the one with the **fewest** number of properties. If there are still multiple possible answers, choose the one that **appear earlier** in the choice.

5. (3 points) Which of the combination of the properties above ensures that the result of `reduce` is always the same regardless of whether it is invoked in parallel or sequentially? Assume that we invoke the `reduce` with `accumulator` assigned to F and `identity` assigned to e.

   (A) Property (1) only.
   (B) Property (1) and (2) only.
   (C) Property (1) and (3) only.
   (D) Property (1), (2), and (3).
   (E) *None of the above*

6. (3 points) Consider the `foldRight` method in your `InfiniteList` class. Assuming that there is an implementation of `foldRight` for stream, given a stream consisting of $[e_1, e_2, e_3, e_4]$, the results of applying `reduce` and `foldRight` with `accumulator` assigned to F and `identity` assigned to e are shown below.

   - `reduce`: $F(F(F(F(e, e_1), e_2), e_3), e_4)$.
   - `foldRight`: $F(e_1, F(e_2, F(e_3, F(e_4, e))))$.

   Let us not consider a parallel version of these and only consider the sequential version of these. Which of the combination of the properties above ensures that the result are always the same?

   (A) Property (1) only.
   (B) Property (1) and (2) only.
   (C) Property (1) and (3) only.
   (D) Property (1), (2), and (3).
   (E) *None of the above*

Continue on the next page.

The next **SIX (6)** questions are MRQ. There can be several correct answers but at least one correct answer. All questions are **independent** of each other. Each question is worth 4 marks. Partial marks **may** be given depending on the number of correct answers in the question and only if the minimum number of correct choices is made.

Shade the bubbles in the answer sheet accordingly. Please use pencil. If you need to change your answer, **completely erase** your previous answer before shading another answer.

7. (4 points) Select ALL the possible definition of the `main` method. Recap that the `main` method is the *entry point* of the program.

   (A) `static void main(String[] args) { .. }`

   (B) `static public void main(String[] str) { .. }`

   (C) `public static int main(String[] args) { .. }`

   (D) `public static void main() { .. }`

   (E) `public static void main(String... args) { .. }`

   (F) `static public final void main(String[] args) { .. }`

8. (4 points) Consider a stream of **odd positive numbers** (*i.e.,* $[1, 3, 5, 7, 9, ...]$). Select ALL expressions that produces a stream of odd positive numbers with infinite number of elements.

   (A) `Stream.iterate(1, i -> i + 2).map(n -> n)`

   (B) `Stream.iterate(0, i -> i + 2).map(n -> n + 1)`

   (C) `Stream.iterate(0, i -> i + 1).map(n -> 2 * n + 1)`

   (D) `Stream.iterate(0, i -> i + 1).filter(n -> 2 * n + 1)`

   (E) `Stream.iterate(0, i -> i + 1).map(n -> n % 2 == 1)`

   (F) `Stream.iterate(0, i -> i + 1).filter(n -> n % 2 == 1)`

Continue on the next page.

For **Question 9 to 10** below, consider the following method. Note that division by zero causes `ArithmeticException`.

```
boolean bar(Integer n) {
  try {
    return 1/n != 0;
  } catch(ArithmeticException e) {
    // Line C
    throw e;
  }
}
int foo(Integer n) {
  try {
    boolean isOne = n.equals(1);
    // Line A
    boolean isBar = bar(n);
    // Line B
    return 1;
  } catch (ArithmeticException e) {
    // Line D
    return 2;
  } catch (NullPointerException e) {
    // Line E
  }
  // Line F;
  return 3;
}
```

9. (4 points) Select ALL the lines that are executed by the expression `foo(0)`.

   (A) Line A
   (B) Line B
   (C) Line C
   (D) Line D
   (E) Line E
   (F) Line F

10. (4 points) Select ALL the lines that are executed by the expression `foo(null)`.

   (A) Line A
   (B) Line B
   (C) Line C
   (D) Line D
   (E) Line E
   (F) Line F

Continue on the next page.

For **Question 11 to 12** below, consider the following classes.

```
class A {
  public String foo(A a) {
    return "1";
  }
  public String foo(C a) {
    return "2";
  }
}
class B extends A {
  public String foo(A a) {
    return super.foo(a) + "3";
  }
  public String foo(B a) {
    return super.foo(a) + "4";
  }
}
class C extends B {
  public String foo(B a) {
    return super.foo(a) + "5";
  }
  public String foo(C a) {
    return super.foo(a) + "6";
  }
}
```

Further consider the following variables:

```
A a = new C();
B b = new C();
C c = new C();
```

Recap that the type of `super` in the method invocation `super.foo(..)` is exactly the type of the superclass as declared in the class declaration.

11. (4 points) Select ALL the methods invoked by the expression `c.foo(a)`?

(A) `A::foo(A)`
(B) `A::foo(C)`
(C) `B::foo(A)`
(D) `B::foo(B)`
(E) `C::foo(B)`
(F) `C::foo(C)`

12. (4 points) Select ALL the method invocation that produces a string of length 3?

(A) `b.foo(a)`
(B) `b.foo(b)`
(C) `b.foo(c)`
(D) `c.foo(a)`
(E) `c.foo(b)`
(F) `c.foo(c)`

Continue on the next page.

Section II: Short Answer

13. (15 points) Consider the following program involving `CompletableFuture`. Assume that each call to `sleep()` causes the thread to sleep for a different arbitrary amount of time. *See API for more information about each methods.*

```
class MultiThreading {
    : // sleep() implementation omitted
  public static String foo() {
    CompletableFuture<String> taskA, taskB, taskC, taskD;

    taskA = CompletableFuture.supplyAsync(() -> {
      sleep();
      System.out.print("A");
      return "A";
    });
    taskB = CompletableFuture.supplyAsync(() -> {
      sleep();
      System.out.print("B");
      return "B";
    });
    taskC = taskA.thenApplyAsync(a -> {
      sleep();
      System.out.print("C");
      return a + "C";
    });
    taskD = taskB.thenCombineAsync(taskA, (b, a) -> {
      sleep();
      System.out.print("D");
      return a + b + "D";
    });
    return taskD.join();
  }

  public static void main(String[] args) {
    foo();
  }
}
```

In this question, if the string `"DCBA"` is a possible answer, then you should write `DCBA` (*without quote*) as one of the answer. Put each answer in its own box. You need not use all the available boxes.

(a) (12 points) Write all the possible strings **printed** by the program. Recap that `System.out.print` does not print a new line.

(b) (3 points) Write all the possible strings **returned** by the function `foo()`.

Continue on the next page.

14. (8 points) We have seen how powerful `flatMap` is by defining `map` in terms of `flatMap` (*i.e.,* `map` *invoke* `flatMap`). Believe it or not, we can also implement `filter` using `flatMap`!

    In this case, we are interested in the implementation of `filter` for `InfiniteList` assuming that the corresponding `flatMap` is already given. The behavior of `flatMap` is summarized by the following usage below.

```
InfiniteList.iterate(1, i -> i + 1)
            .flatMap(x -> InfiniteList.generate(x).limit(x)
   );

// after iterate: [1, 2,    3,       4,          ...]
// after flatMap: [1, 2, 2, 3, 3, 3, 4, 4, 4, 4, ...]


InfiniteList.sentinel()
            .flatMap(x -> InfiniteList.generate(x).limit(x)
   );

// after iterate: []
// after flatMap: []
```

    The skeleton for the `filter` is shown below and we have **THREE (3)** blanks. Fill in each of the blanks in the answer sheet with the corresponding **simple expression** (*i.e., no conditionals, loops, lambdas, blocks, etc*). You may invoke the available method in `InfiniteList`.

```
public InfiniteList<T> filter(BooleanCondition<T> pred) {
  return this.flatMap(elem -> {
    if (__Blank 1__) {     // part (a)
      return __Blank 2__;  // part (b)
    } else {
      return __Blank 3__;  // part (c)
    }
  });
}
```

    NOTE: The distribution of marks depends on the condition used in `__Blank 1__`.

Continue on the next page.

15. (8 points) Consider the following nested class.

```
interface I {
  void bar(int x);
}

class A {
  int x = 1; /* (1) */

  class B {
    private int x = 2; /* (2) */

    public int getX() {
      return this.x;
    }
  }

  I foo(int x) { /* (3) */
    class C extends B implements I {
      private int x = 4; /* (4) */

      public int getX() {
        return this.x;
      }

      public void bar(int x) { /* (5) */
        System.out.println(__Blank__);
      }
    }
    return new C();
  }
}
```

We have **FIVE (5)** different version of the name `x`. This may be a field or a variable. We want to print each version of `x` in `System.out.println(__Blank__)`. For each of the version, write an expression that can be used to print that version of `x`. If it is not possible to print the given version of `x`, simply write "***impossible***" (*without quotes*).

Version (5) can be accessed simply by `System.out.println(x)`. This has already been answered for you on the answer sheet. Follow the same format when answering for other versions.

Continue on the next page.

Section III: Long Answer

16. (12 points) The `final`, `private`, and `abstract` keywords can be applied in multiple places. We will now consider a possible usage that you may not have thought of about these keyword.

    For each of the scenario given, determine if the code can compile or not. Answer this on the line "**Does `Code #N` Compile**" with either **YES** (*if the code compiles*) or **NO** (*if the code does not compile*). Then explain your answer **in the box** in no more than 3 sentences.

    (a) (4 points) Code #1

```
final abstract class C1 {
  private static final int VAL = 1;

  public static int foo() {
    return VAL;
  }
}
```

    (b) (4 points) Code #2

```
abstract class C2 {
  private abstract int foo();
}
```

    (c) (4 points) Code #3

```
abstract class C3 {
  private final int x;

  private C3(int x) {
    if(x < 0) {
      this.x = -x;
    } else {
      this.x = x;
    }
  }
}
```

Continue on the next page.

17. (15 points) Consider the following attempt at creating a monad.

```
class Box {
  private final int val;
  private final int ctx;

  // Private Constructor
  private Box(int val, int ctx) {
    this.val = val;
    this.ctx = ctx;
  }

  // Factory Method
  public static Box of(int val) {
    return new Box(val, 0);
  }
  public Box flatMap(Transformer<Integer, Box> fn) {
    return Box.merge(this, fn.transform(this.val));
  }
  public static Box merge(Box b1, Box b2) {
    return new Box(b2.val, (b1.ctx + (b1.ctx * b2.ctx)));
  }

  // Other Available Functions
  public static Box f1(int val) {
    return new Box(val * val, -val);
  }
  public static Box f2(int val) {
    return new Box(val - 10, -val);
  }
}
```

For each of the law for monad and functor, check if `Box` satisfies the law. Answer **YES** if `Box` satisfies the law and answer **NO** otherwise. If the answer is **NO**, provide a simple counter-example by specifying monads of the form `{val = n; ctx = c}` and referencing only the two static methods `f1` and `f2`.

(a) (5 points) Does `Box` satisfy the **Left Identity Law** of Monad.

$\forall$ `x, f: Monad.of(x).flatMap(y -> f(y))` $\equiv$ `f(x)`

(b) (5 points) Does `Box` satisfy the **Right Identity Law** of Monad.

$\forall$ `monad: monad.flatMap(x -> Monad.of(x))` $\equiv$ `monad`

(c) (5 points) Does `Box` satisfy the **Associative Law** of Monad.

$\forall$ `monad, f, g: monad.flatMap(x -> f(x)).flatMap(y -> g(y))`
$\equiv$
$\forall$ `monad, f, g: monad.flatMap(x -> f(x).flatMap(y -> g(y)))`

End of questions.

This page is intentionally left blank. You may use this for your working. Answers written here will not be graded.

API

Stream<T>

Data Sources

- `static <T> Stream<T> of(T... values)`
  Returns a sequential ordered stream whose elements are the specified values.

- `static <T> Stream<T> generate(Supplier<? extends T> s)`
  Returns an infinite sequential unordered stream where each element is generated by the provided `Supplier`.

  – Also available in `InfiniteList`.

- `static <T> Stream<T> iterate(T seed, Predicate<? super T> hasNext, UnaryOperator<T> next)`
  Returns a sequential ordered `Stream` produced by iterative application of the given `next` function to an initial element, conditioned on satisfying the given `hasNext` predicate.

- `static <T> Stream<T> iterate(T seed, UnaryOperator<T> f)`
  Returns an infinite sequential ordered `Stream` produced by iterative application of a function `f` to an initial element seed, producing a `Stream` consisting of `seed`, `f(seed)`, `f(f(seed))`, etc.

  – Also available in `InfiniteList`.

- To convert from `List` named `list` to `Stream`, you can invoke `list.stream()`.

Intermediate Operations

- `Stream<T> dropWhile(Predicate<? super T> predicate)`
  Returns, if this stream is ordered, a stream consisting of the remaining elements of this stream after dropping the longest prefix of elements that match the given predicate.

- `Stream<T> takeWhile(Predicate<? super T> predicate)`
  Returns, if this stream is ordered, a stream consisting of the longest prefix of elements taken from this stream that match the given predicate.

  – Also available in `InfiniteList`.

- `<R> Stream<R> map(Function<? super T, ? extends R> mapper)`
  Returns a stream consisting of the results of applying the given function to the elements of this stream.

  – Also available in `InfiniteList`.

- `Stream<T> filter(Function<? super T, ? extends R> mapper)`
  Returns a stream consisting of the elements of this stream that match the given predicate.
  - Also available in `InfiniteList`.
- `<R> Stream<R> flatMap(`
  `Function<? super T, ? extends Stream<? extends R>> mapper)`
  Returns a stream consisting of the results of replacing each element of this stream with the contents of a mapped stream produced by applying the provided mapping function to each element.
- `Stream<T> limit(long maxSize)`
  Returns a stream consisting of the elements of this stream, truncated to be no longer than `maxSize` in length.
  - Also available in `InfiniteList`.
- `Stream<T> peek(Consumer<? super T> action)`
  Returns a stream consisting of the elements of this stream, additionally performing the provided action on each element as elements are consumed from the resulting stream.

Terminal Operations

- `boolean allMatch(Predicate<? super T> predicate)`
  Returns whether all elements of this stream match the provided predicate.
- `boolean anyMatch(Predicate<? super T> predicate)`
  Returns whether any elements of this stream match the provided predicate.
- `boolean noneMatch(Predicate<? super T> predicate)`
  Returns whether no elements of this stream match the provided predicate.
- `long count()`
  Returns the count of elements in this stream.
  - Also available in `InfiniteList`.
- `void forEach(Consumer<? super T> action)`
  Performs an action for each element of this stream.
  - Also available in `InfiniteList`.
- `List<T> toList()`
  Accumulates the elements of this stream into a `List`.
  - Also available in `InfiniteList`.
- `T reduce(T identity, BinaryOperator<T> accumulator)`
  Performs a reduction on the elements of this stream, using the provided identity value and an associative accumulation function, and returns the reduced value.

- `<U> U reduce(U identity,`
  `BiFunction<U, ? super T, U> accumulator,`
  `BinaryOperator<U> combiner)`
  Performs a reduction on the elements of this stream, using the provided identity, accumulation and combining functions.
  - What is available in `InfiniteList` is `foldRight` which is similar to `Reduce` except the order is from right-to-left.

CompletableFuture<T>

Creation

- `static <U> CompletableFuture<U> supplyAsync(Supplier<U> supplier)`
  Returns a new `CompletableFuture` that is asynchronously completed by a task running in the `ForkJoinPool.commonPool()` with the value obtained by calling the given `Supplier`.
- `static <U> CompletableFuture<U> completedFuture(U value)`
  Returns a new `CompletableFuture` that is already completed with the given value.

Operations

- `<U> CompletableFuture<U> thenApplyAsync(`
  `Function<? super T, ? extends U> fn)`
  Returns a new `CompletionStage` that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage's result as the argument to the supplied function.
  - Similar to `map(fn)`.
- `<U> CompletableFuture<U> thenComposeAsync(`
  `Function<? super T, ? extends CompletionStage<U>> fn)`
  Returns a new `CompletionStage` that is completed with the same value as the `CompletionStage` returned by the given function, executed using this stage's default asynchronous execution facility.
  - Similar to `flatMap(fn)`.
- `<U, V> CompletableFuture<V> thenCombineAsync(`
  `CompletionStage<? extends U> other,`
  `BiFunction<? super T, ? super U, ? extends V> fn)`
  Returns a new `CompletionStage` that, when this and the other given stage both complete normally, is executed using this stage's default asynchronous execution facility, with the two results as arguments to the supplied function.

Waiting

- `static CompletableFuture<Void> allOf(CompletableFuture<?>... cfs)`
  Returns a new `CompletableFuture` that is completed when all of the given `CompletableFuture` complete.
- `static CompletableFuture<Object> anyOf(CompletableFuture<?>... cfs)`
  Returns a new `CompletableFuture` that is completed when any of the given `CompletableFuture` complete, with the same result.
- `T join()`
  Returns the result value when complete, or throws an (unchecked) exception if completed exceptionally.

– END OF PAPER –